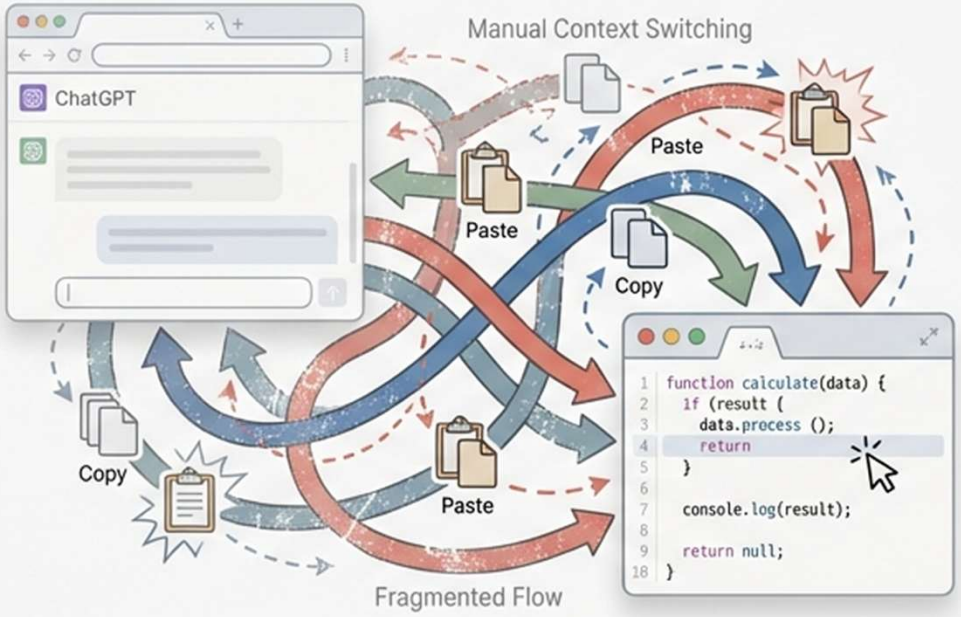
Gemini CLI: AI 페어 프로그래머

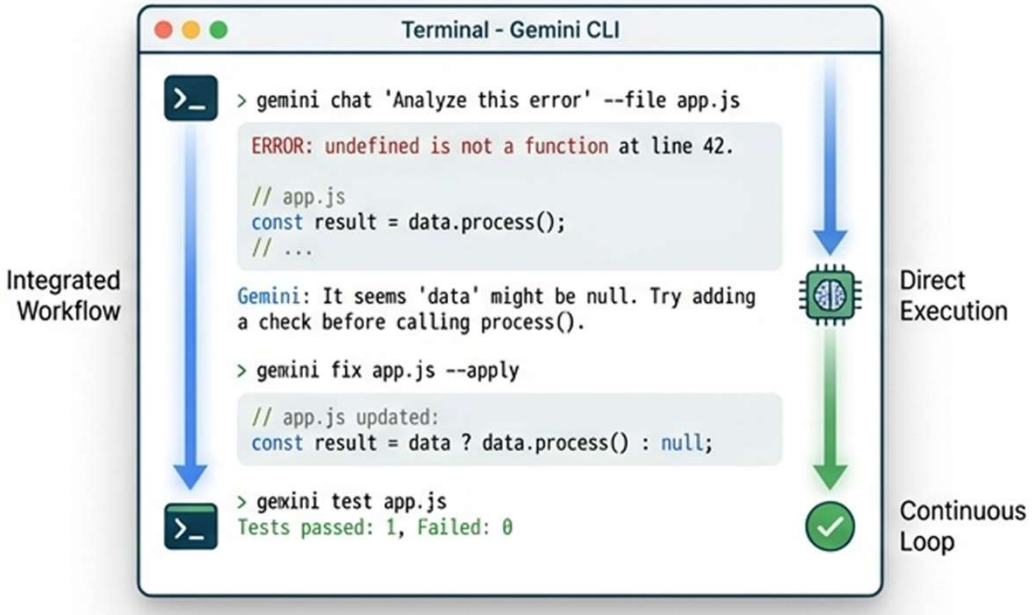
터미널에서 만나는 강력한 코딩 파트너

Sung-Dong Kim,
School of Computer Engineering,
Hansung University

도입 - 왜 Gemini CLI인가?

터미널을 떠나지 않는 몰입형 개발 경험





- **웹 UI의 한계 극복**
  - 복사-붙여넣기(Copy & Paste)의 반복 작업 제거
  - 창 전환(Context Switching) 없이 코딩 흐름 유지

- **개발 워크플로우(Workflow)의 통합**
  - 코드 작성 → 에러 확인 → 즉시 질의(AI) → 수정의 단일 루프
- **자동화의 확장성**
  - 단일 파일 분석부터 유닛 테스트 생성까지 명령어로 해결

Gemini CLI 시작하기

3단계로 끝나는 개발 환경 설정

01

API 키 발급

Google AI Studio 접속 → Create API key

02

도구 설치

Node.js 환경 확인 (node -v)

```
npm install -g @google/gemini-cli
```

03

인증 설정

환경 변수 설정

```
export GEMINI_API_KEY='My_Key'
```

운영체제별 환경 변수 설정

Windows (PowerShell)

```
[System.Environment]::SetEnvironmentVari
able('GEMINI_API_KEY', '키_입력', 'User')
```

설정 후 터미널 재시작 필수

macOS / Linux

```
export GEMINI_API_KEY='발급받은_키_입력'
```

.zshrc 또는 .bashrc 파일에 추가

연결 확인

```
gemini '안녕! 오늘부터 내 코딩 파트너가 되어줘'
```

기본 명령어 익히기

단순 질의

```
gemini '질문 내용'
```

일반적인 기술 질문 및 개념 학습

파일 설명

```
gemini explain <파일영>
```

파일 내용을 읽고 동작 원리 요약

파일 기반 질의

```
gemini '질문' -f <파일>
```

소스 코드를 컨텍스트로 전달 (-f 옵션)

첫 번째 대화

프로젝트 생성부터 실행까지

1. 새 디렉토리 생성
2. Gemini 실행
3. 자연어로 명령

```
$ mkdir gemini_proj
$ cd gemini_proj
$ gemini
User: 안녕하세요!
Gemini: 안녕하세요! 어떤 코딩 작업을 도와드릴까요?
User: 2~9단 C언어 구구단 프로그램을 만들어줘.
Gemini: 네, 구구단 코드를 작성해 보겠습니다...
```

효과적인 프롬프트 공식

페르소나 (Persona) + 작업 (Task) + 맥락 (Context) + 형식 (Format) = 성공 (Success)

✗ 코드 짜줘.

✓ 너는 백엔드 전문가야(Persona).
사용자 인증을 처리하는 Express.js 미들웨어를 작성해줘(Task).
JWT 방식을 사용하고(Context) , 단계별로 설명해줘(Format).

실습 1: C 언어 디버깅

메모리 누수 찾기 (buggy_list.c)

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void add_node(struct Node** head, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;
}
int main() {
    struct Node* head = NULL;
    add_node(&head, 10);
    printf("Head data: %d\n", head->data);
    return 0;
}
```

Gemini의 분석 결과

Analysis Command

```
$ gemini analyze buggy_list.c
'이 코드에서 메모리 누수 지점을 찾고 수정안을 줘.'
```

Security Review Command

```
$ gemini review buggy_list.c
'보안상 위험한(Segmentation Fault) 부분이 있어?'
```

Result: AI가 free() 함수 누락과 잠재적 Segmentation Fault 위험을 감지하고 수정 코드를 제안합니다.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void add_node(struct Node** head, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;
}

int main() {
    struct Node* head = NULL;
    add_node(&head, 10);
    add_node(&head, 20);
    printf("Head data: %d\n", head->data);
    // 의도적 실수: 메모리 해제(free)를 하지 않음
    return 0;
}
```

C 언어 메모리 누수 탐지

연결 리스트(Linked List) 구현 중
발생하는 흔한 실수 디버깅

Prompt Action

`gemini analyze buggy_list.c "이 코드에서 메모리 누수가 발생하는 지점을 찾고 수정해줘."`

malloc 할당 후 free 누락

실습 2: 파이썬 리팩토링

코드 다이어트 (Code Diet)

Before

```python
# Before: process_data.py
def get_even_squares(numbers):
    result = []
    for n in numbers:
        if n % 2 == 0:
            square = n * n
            result.append(square)
    return result

data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(get_even_squares(data))
```

Python 코드 다이어트: 리팩토링

복잡한 루프를 'Pythonic'하고 간결한 코드로 변환하기

```python
def get_even_squares(numbers):
    result = []
    for n in numbers:
        if n % 2 == 0:
            square = n * n
            result.append(square)
    return result

data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(get_even_squares(data))
```

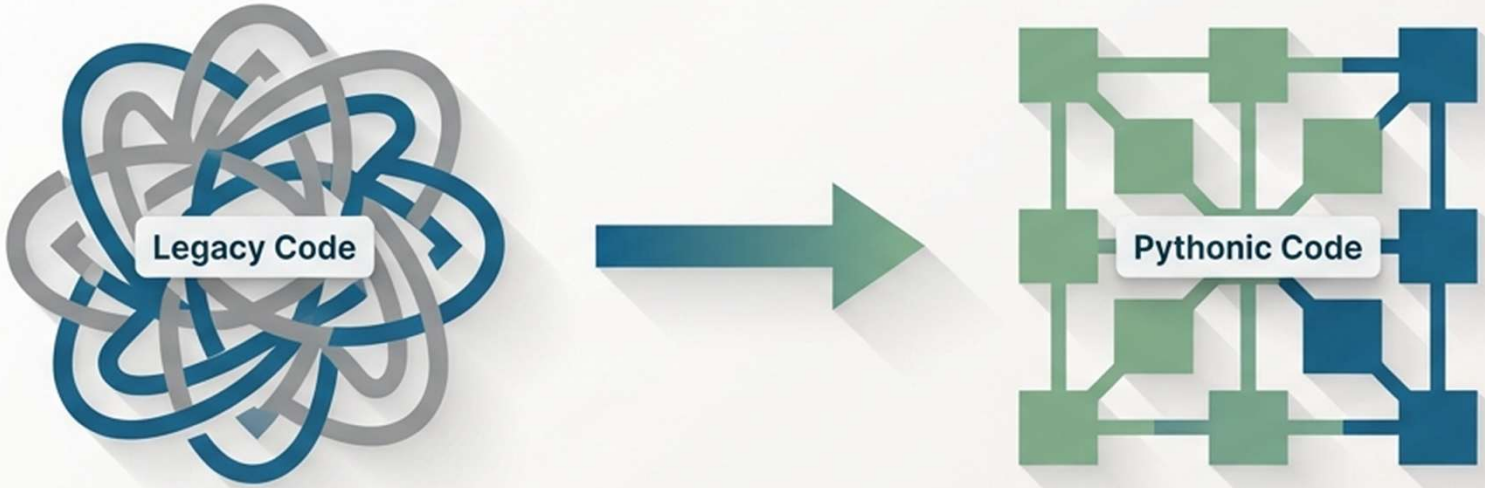
Refactoring

```
gemini "List Comprehension을 사용해 한 줄로 리팩토링해줘." -f process_data.py
```

Documentation

```
gemini "Google 스타일의 Docstring을 추가해줘." -i process_data.py
```

[실습] Python 리팩토링과 문서화



코드 다이어트 (Pythonic Code)

복잡한 루프를 간결한 List Comprehension으로 변환

```
gemini '리팩토링해줘' -f process_data.py
```

문서화 자동화 (Documentation)

협업을 위한 Docstring 자동 생성 (-i 옵션: 즉시 수정)

```
gemini 'Google 스타일 Docstring 추가해줘' -i process_data.py
```

코드 검증 및 언어 확장

작성한 로직의 신뢰성 확보와 타 언어로의 변환

자동 테스트 생성 (Verification)

수동 테스트 대신 pytest 자동 생성

```
gemini "pytest를 사용해서 이 코드의 테스트 케이스를 짜줘." -f
process_data.py > test_process_data.py
```

언어 변환 (Expansion)

C언어 포인터 구조 → Python 클래스 객체 변환

```
gemini "buggy_list.c의 구조를 Python 클래스로 변환하고 주석 달아줘."
```

단순 채팅을 넘어선 기능

/memory

대화 맥락(Context)
관리 및 기억

-f <file>

파일 직접 읽기 및 분석

/chat

대화 상태 저장 (Save)
및 불러오기 (Resume)

/copy

답변 코드를
클립보드로 복사

Gemini CLI 치트시트

동작 (Action)	명령어 (Command)
설치	`npm install -g @google/gemini-cli`
실행	`gemini`
파일 분석	`gemini -f <file> "instruction"`
관리	`/chat, /memory, /init`
시스템	`/about, /help`

단계별 코드 리뷰: 'Hello World' 다시 보기

단순 작동을 넘어 전문가 수준의 코드로 발전시키기

단순 오류 수정

```
gemini "이 코드가 왜 컴파일 오류가
나는지 알려주고 수정해줘" -f hello.c
```

스타일 및 컨벤션

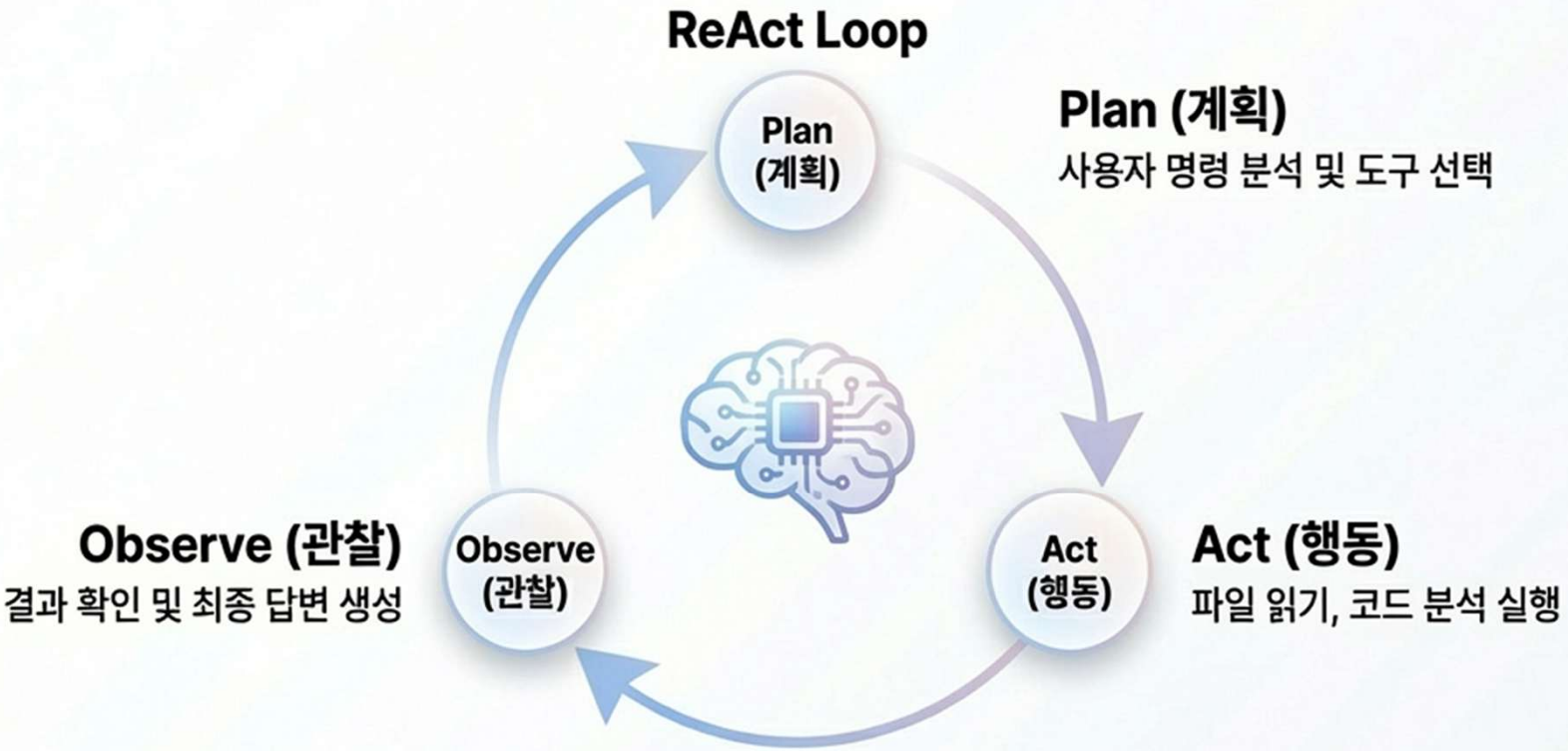
```
gemini "이 코드를 C99 표준(또는
PEP 8)에 맞게 수정해줘" -f hello.c
```

원리 이해 (Deep Dive)

```
gemini "이 프로그램이 메모리에서
어떻게 실행되는지 설명해줘"
```

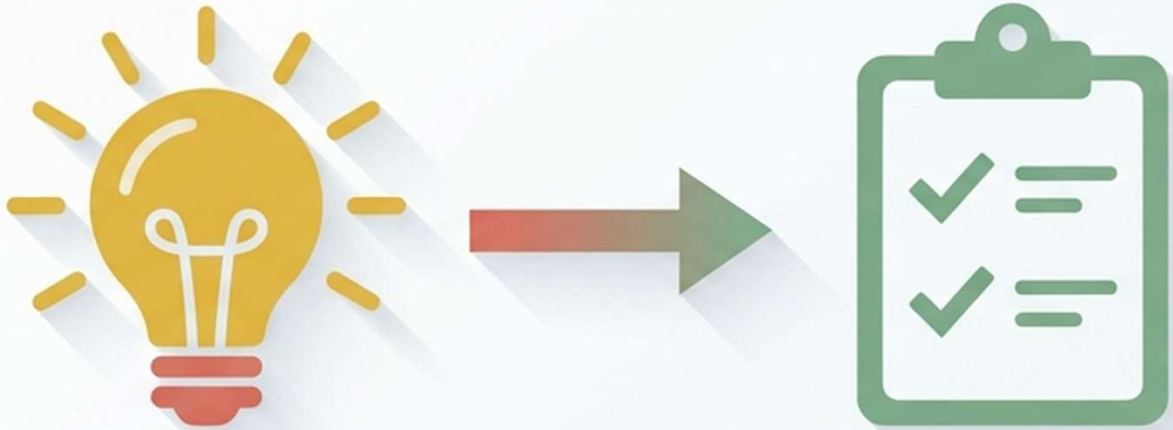
단순 채팅이 아닌 '에이전트(Agent)'

Gemini가 터미널에서 스스로 생각하고 행동하는 방식 (ReAct Loop)



AI가 스스로 파일 시스템에 접근하여 맥락을 파악

효과적인 AI 활용 가이드



프롬프트 디테일 (Detail is Key)

- 단순 요청보다 구체적인 조건 부여 (예: 메모리 효율, 초보자용 설명)

비판적 수용 (Human in the Loop)

- AI 제안 코드의 실행 여부 반드시 검증 (Blind Copy 지양)
- 오류 발생 시 재질문(Follow-up) 필수

```
gemini '방금 코드는 에러가 나는데? 다시 확인해줘'
```

학습 도구로 활용

- 단순 정답 요구보다 '왜?'(Why)를 질문하여 원리 이해